17. Using Spring Template manage Database and Transaction

```
CREATE DATABASE springdb;
USE springdb;
CREATE TABLE employee (
   id INT PRIMARY KEY,
   name VARCHAR(50),
   salary DOUBLE);
<dependencies>

   <!-- Spring JDBC -->
   <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-jdbc</artifactId>
      <version>6.1.10</version>
   </dependency>
   <!-- Spring Transaction -->
   <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-tx</artifactId>
      <version>6.1.10</version>
   </dependency>
   <!-- MySQL JDBC Driver -->
   <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
```

```
        <version>9.0.0</version>
    </dependency>
</dependencies>
@Configuration
@EnableTransactionManagement
public class AppConfig {

    @Bean
    public DataSource dataSource() {

        DriverManagerDataSource ds =
                new DriverManagerDataSource();

        ds.setDriverClassName("com.mysql.cj.jdbc.Driver");
        ds.setUrl("jdbc:mysql://localhost:3306/springdb");
        ds.setUsername("root");
        ds.setPassword("root");
        return ds;
    }
    @Bean
    public JdbcTemplate jdbcTemplate() {
        return new JdbcTemplate(dataSource());
    }
    @Bean
    public PlatformTransactionManager transactionManager() {
```

```
        return new DataSourceTransactionManager(dataSource());
    }}
public class Employee {
    private int id;
    private String name;
    private double salary;
    // Getters and Setters
}
@Repository
public class EmployeeDAO {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    public int save(Employee emp) {

        String sql =
        "INSERT INTO employee VALUES(?,?,?)";

        return jdbcTemplate.update(
                sql,
                emp.getId(),
                emp.getName(),
                emp.getSalary());
    }
```

```
    public int update(Employee emp) {

        String sql =
        "UPDATE employee SET name=?, salary=? WHERE id=?";

        return jdbcTemplate.update( sql,
                emp.getName(),
                emp.getSalary(),
                emp.getId());
    }

    public int delete(int id) {
        return jdbcTemplate.update(
                "DELETE FROM employee WHERE id=?", id);
    }
}

@Service
public class EmployeeService {
    @Autowired
   EmployeeDAO dao;
    @Transactional
    public void addEmployee(Employee emp) {
        dao.save(emp);
```

```
        // If any exception occurs here,
        // the transaction will be rolled back.
    }
}
public class Main {

    public static void main(String[] args) {

        AnnotationConfigApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);
        EmployeeService service =
            context.getBean(EmployeeService.class);
        Employee emp = new Employee();
        emp.setId(101);
        emp.setName("Rahul");
        emp.setSalary(50000);
        service.addEmployee(emp);
        System.out.println("Employee Inserted Successfully");
        context.close();
    }
}
```